MANAGING CONFIGURATION AND PERSISTENT STORAGE

CONFIGMAPS

1. A ConfigMap is used to store configuration data in key-value format.
2. It helps you pass configuration values (like environment variables, application settings, URLs, ports, etc.) to Pods and Containers.
3. ConfigMaps are useful because you can change configuration without rebuilding the Docker image.
4. The same ConfigMap can also be shared across multiple Pods.
5. Generally, ConfigMaps are used for setting variables and application configuration in Kubernetes.

1. vim configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
  labels:
    app: myapp
    env: devops
data:
  mysql_root_password: root123
  mysql_user: admin
  mysql_password: admin123
  mysql_database: student
```

2. vim cmss.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: my-ss
  labels:
    app: myapp
    env: devops

spec:
  replicas: 2
  selector:
    matchLabels:
```

```
    app: myapp
  template:
    metadata:
      labels:
        app: myapp
        env: devops
    spec:
      containers:
      - name: mysql-container
        image: mysql:8.0
        ports:
        - name: mysql-port
          containerPort: 3306
        env:
        - name: MYSQL_ROOT_PASSWORD
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: mysql_root_password
        - name: MYSQL_USER
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: mysql_user
        - name: MYSQL_PASSWORD
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: mysql_password
        - name: MYSQL_DATABASE
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: mysql_database
```

1. kubectl apply -f config.yaml
2. kubectl apply -f cmss.yaml
3. kubectl get cm

4. kubectl describe cm my-config
5. kubectl get pods

SECRETS

1. Secret is a Kubernetes object used to store sensitive information securely.
2. It stores data like:
    1. passwords
    2. usernames
    3. API keys
    4. tokens
    5. database credentials
3. Secrets help avoid hardcoding sensitive data inside Pod YAML files or container images.
4. Secret values are stored in Base64 encoded format.

Why We Use Secrets

- To protect sensitive data
- To separate confidential information from application code
- To manage credentials securely across multiple Pods

1.vim secrets.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret
  labels:
    app: myapp
    env: devops
data:
  mysql_root_password: cm9vdDEyMw==
  mysql_user: YWRtaW4=
  mysql_password: YWRtaW4xMjM=
  mysql_database: c3R1ZGVudA==
```

2. vim secretstatefulset.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: my-ss
```

```
  labels:
    app: myapp
    env: devops
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
        env: devops
    spec:
      containers:
      - name: mysql-container
        image: mysql:8.0
        ports:
        - name: mysql-port
          containerPort: 3306
        env:
        - name: MYSQL_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mysql-secret
              key: mysql_root_password
        - name: MYSQL_USER
          valueFrom:
            secretKeyRef:
              name: mysql-secret
              key: mysql_user
        - name: MYSQL_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mysql-secret
              key: mysql_password
        - name: MYSQL_DATABASE
          valueFrom:
```

```
      secretKeyRef:
        name: mysql-secret
        key: mysql_database
```

1. kubectl apply -f secret.yaml
2. kubectl apply -f secretstatefulset.yaml
3. kubectl get secrets
4. kubectl describe secrets mysql-secret
5. kubectl get pods
6. kubectl exec -it my-ss-0 -- bash
    1. mysql -u admin -h localhost -padmin123
    2. show databases;

Note:

1. echo -n “admin” | base64 ----------> to encrypt data in base64 format
2. echo -n "YWRtaW4=" | base64 --decode -----> to decrypt the password

---

PERSISTENT VOLUME

Persistent Volume is a cluster storage resource in Kubernetes used to store application data permanently even if the pod is deleted or restarted.

WHY WE USE PERSISTENT VOLUME

We use PV for applications that need permanent data storage like:

- MySQL
- MongoDB
- PostgreSQL
- Jenkins
- WordPress

ACCESS MODES IN PERSISTENT VOLUME

1. ReadWriteOnce (RWO)
2. ReadOnlyMany (ROX)
3. ReadWriteMany (RWX)

1. ReadWriteOnce (RWO)

One node can read and write the volume.

Most commonly used with:

- MySQL

- PostgreSQL
- MongoDB

Suppose:
- Node-1 has MySQL pod
- PV attached to Node-1

1. Only Node-1 can use that storage.
2. Other nodes cannot use it at the same time.

But
1. Multiple pods CAN use it
2. BUT only if pods are running on SAME node.

2. ReadOnlyMany (ROX)

ROX simply means Multiple nodes can READ but nobody can write.

Suppose:
- 10 web servers
- All need same image files

- All servers can read files together.
- But cannot modify files.

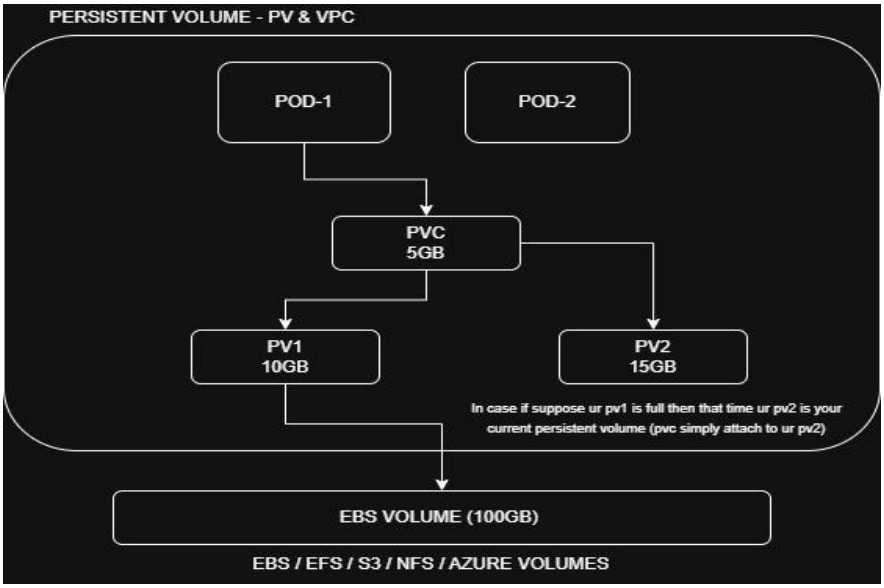
3. ReadWriteMany (RWX)

RWX means Multiple nodes can READ and WRITE together.

Used in:
- Shared file systems
- Team shared storage
- Logging systems

Suppose:
- 3 application pods
- All need to store logs in same storage

All can write together.



TYPES OF RECLAIM POLICIES IN PERSISTENT VOLUME

1. Retain
2. Recycle
3. Delete

It simply decides What happens to the storage after PVC is deleted

1. RETAIN

- Data remains safe
- PV remains in cluster
- Kubernetes does not delete data
- Administrator must manually clean storage.

PVC Deleted ----------> PV remains ---------> data remains safe

Suppose:

- MySQL pod deleted
- PVC deleted

Still:
Database files remain safe in PV

2. RECYCLE

- Old data removed
- PV becomes available again
- Kubernetes cleans the volume using basic rm -rf.

PVC Deleted -------> Data cleaned --------> PV reused

3. DELETE

- PVC deleted
- PV deleted
- Actual storage deleted automatically

Mostly used in:

- Cloud storage
- AWS EBS
- Azure Disk
- GCP Disk

PVC Deleted ---------> PV Deleted ---------> Storage Deleted

Suppose:

- AWS EBS volume attached
- PVC deleted

Then:
Kubernetes automatically deletes EBS disk.

1.vim pv.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mypv
spec:
  capacity:
    storage: 10Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: standard
  hostPath:
    path: /mnt/data
```

2.vim pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mypvc
spec:
  storageClassName: standard
```

```
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

3.vim pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
  labels:
    app: myapp
spec:
  volumes:
  - name: my-vol
    persistentVolumeClaim:
      claimName: mypvc
  containers:
  - name: myapp
    image: nginx:latest
    ports:
    - containerPort: 80
    volumeMounts:
    - mountPath: /usr/share/nginx/html/
      name: my-vol
```

1. kubectl apply -f pv.yaml
2. kubectl apply -f pvc.yaml
3. kubectl apply -f pod.yaml
4. kubectl get pv
5. kubectl get pvc

TO CHECK DATA IN WORKER-NODE

6. ssh node01
7. ls /mnt/data

TRY TO DELETE POD AND AGAIN CREATE POD

1.kubectl delete -f pod.yaml
2.kubectl apply -f pod.yaml

3.kubectl get pods -o wide
4.curl <ip_of_pod>

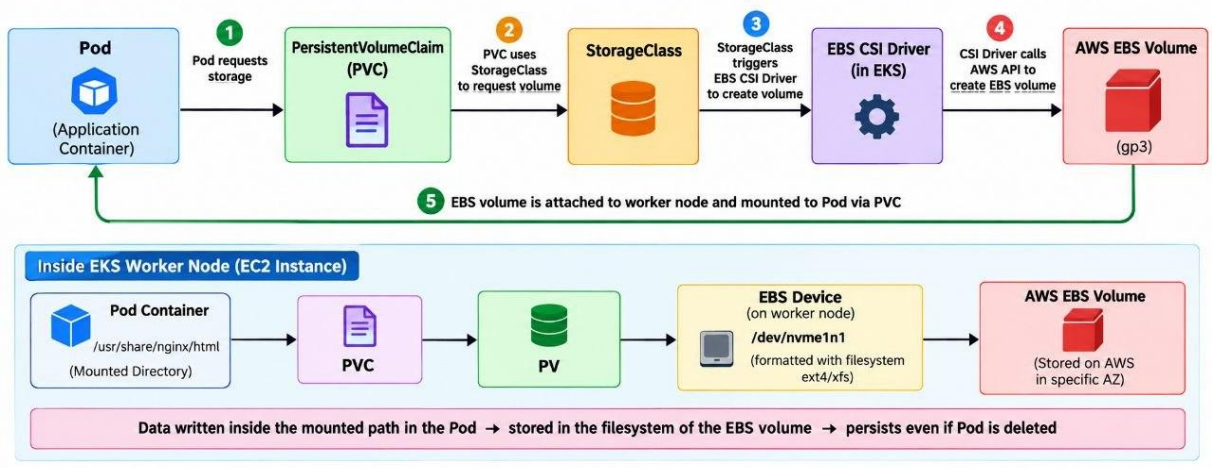
NOTE:

KILLERKODA

1. If we are working on KillerKoda and using hostPath volume, then Kubernetes stores data inside the node filesystem like /mnt/data.
2. To check the actual stored data inside /mnt/data, we must enter into the node
3. In KillerKoda, we can enter into the worker node using: ssh node01
4. After entering the node, verify the data:
5. cd /mnt/data
6. ls

- if we work on EBS then access type is ReadWriteOnce (RWO)
- if we work on EFS then access type is ReadWriteMany (RWX)

COMPLETE EKS PERSISTENT VOLUME PRACTICAL WITH EBS



1.CHECK THE CLUSTER

1. aws sts get-caller-identity
2. aws eks update-kubeconfig --region us-west-2 --name eks
3. kubectl get nodes

2.CHECK OIDC PROVIDER

1.Check

```
aws eks describe-cluster \
  --name eks \
  --region us-west-2 \
  --query "cluster.identity.oidc.issuer" \
  --output text
```

2. Now check OIDC provider is associated:

```
eksctl utils associate-iam-oidc-provider \
  --region us-west-2 \
  --cluster eks \
  --approve
```

3. INSTALL EBS CSI DRIVER

1.Check Existing add-on

```
aws eks list-addons \
  --cluster-name eks \
  --region us-west-2
```

2.Install

```
eksctl create addon \
  --name aws-ebs-csi-driver \
  --cluster eks \
  --region us-west-2 \
  --force
```

3.Then check active or not

```
aws eks describe-addon \
  --cluster-name eks \
  --addon-name aws-ebs-csi-driver \
  --region us-west-2 \
  --query "addon.status"
```

4. CHECK EBS CSI PODS

1. Check ebs csi pods

```
kubectl get pods -n kube-system | grep ebs
```

Expected:

ebs-csi-controller-xxxxx Running
ebs-csi-controller-xxxxx Running
ebs-csi-node-xxxxx Running

2.Check DaemonSet
kubectl get daemonset -n kube-system | grep ebs

3.Check Deployment
Kubectl get deployment -n kube-system | grep ebs

5. IAM PERMISSIONS
The EBS CSI controller needs AWS permissions to perform operations such as:

1. CreateVolume
2. DeleteVolume
3. AttachVolume
4. DetachVolume
5. DescribeVolume

For a modern EKS cluster, configure the EBS CSI add-on with an IAM role rather than manually creating the service account.

If your cluster/add-on setup requires an IAM service account, first associate OIDC:
1.Check
eksctl utils associate-iam-oidc-provider \
 --region us-west-2 \
 --cluster eks \
 --approve

2.Create
eksctl create iamserviceaccount \
 --name ebs-csi-controller-sa \
 --namespace kube-system \
 --cluster eks \
 --region us-west-2 \
 --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy \
 --approve \
 --override-existing-serviceaccounts

3.Check Service Account
kubectl get sa ebs-csi-controller-sa -n kube-system

6.CREATE STORAGECLASS
1.vim storageclass.yaml
```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-sc
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
parameters:
  type: gp3
reclaimPolicy: Delete
```

2. kubectl apply -f storageclass.yaml
3. kubectl get storageclass

7.CREATE PVC
1.vim pvc.yaml
```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mypvc
spec:
  storageClassName: ebs-sc
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

2.kubectl apply -f pvc.yaml
3.kubectl get pvc

Note:
1. *If pvc is in pending state These is normal because we used*

volumeBindingMode: WaitForFirstConsumer

2. *Kubernetes waits until a Pod uses the PVC so it can choose an appropriate node/AZ for the EBS volume.*

8.CREATE A POD

1.vim pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
  labels:
    app: myapp
spec:
  containers:
    - name: myapp
      image: nginx:latest
      volumeMounts:
        - name: myvol
          mountPath: /usr/share/nginx/html

  volumes:
    - name: myvol
      persistentVolumeClaim:
        claimName: mypvc
```

2.kubectl apply -f pod.yaml

9.CHECK ALL THE MANIFESTS

1. kubectl get pods
2. kubectl get pvc
3. kubectl get storageclass

10.CHECK AWS CONSOLE

AWS Console → EC2 → Elastic Block Store → Volumes

You should get a newly created 5GB volume

11.WRITE DATA IN EBS

1. kubectl exec -it mypod – bash
   echo "HELLO FROM AWS EBS" > /usr/share/nginx/html/index.html
2. curl localhost

12.TEST PERSISTENCE

1. kubectl delete pod mypod
2. kubectl get pods
3. kubectl apply -f pod.yaml
4. kubectl get pods

5. kubectl exec -it mypod – bash
   Curl localhost

IMPORTANT:

EKS add-on

An EKS add-on is an AWS-supported Kubernetes component that you can install and manage in your EKS cluster.

It's a component that helps EKS to communicate with EBS volume

EKS Cluster

1. VPC CNI Add-on
2. CoreDNS Add-on
3. kube-proxy Add-on
4. EBS CSI Driver Add-on

EBS CSI DRIVER

EBS (Elastic Block Storage)

EBS simply means a hard disk attached to an EC2 instance

CSI (Container Storage Interface)

1. CSI is a Container Storage Interface
2. CSI is a standard interface that allows Kubernetes to communicate with different storage systems.

CSI -----> Standard way for Kubernetes to communicate with storage

EBS CSI Driver

EBS CSI Driver is the Kubernetes component that allows Kubernetes to create, attach, mount, and manage AWS EBS volumes.

Without the EBS CSI Driver:

PVC
↓
Kubernetes says: "I need storage"
↓
Kubernetes doesn't know how to create AWS EBS

With the EBS CSI Driver:

PVC
↓
StorageClass
↓
EBS CSI Driver
↓
AWS API
↓
EBS Volume created

OIDC PROVIDER

OIDC allows Kubernetes service accounts to securely get AWS permissions through IAM roles.

OIDC - OpenID Connect

Kubernetes Service Account

|

OIDC

|

AWS IAM

|

AWS Permissions

WHY DO WE NEED IT

The EBS CSI Driver needs to perform AWS operations such as:

1. Create EBS volume
2. Delete EBS volume
3. Attach EBS volume
4. Detach EBS volume
5. Describe EBS volume

SERVICE ACCOUNT

A Kubernetes ServiceAccount is an identity used by Pods.

Flow:
Pod --------- Service Account ---------- Iam Role -------- AWS Permissions

So instead of giving AWS credentials directly to the Pod, we associate the Kubernetes ServiceAccount with an AWS IAM role.
